CPU/GPU/NPU-Aware RAG Pipeline on an Intel Core Ultra: Phase 1 Measurement and Profiling Report

Ting-Yu Yu, Cheng-Yang Wu, Jonas Tsai, Vincent Chen

CSE 291A, Fall 2026 – Phase 1 (due Nov 4)

1 Goal and summary

Our project asks where each stage of an edge RAG pipeline should run on a machine whose CPU, integrated GPU (iGPU) and NPU share one DRAM. The pipeline is: query → **S1** embedding (bge-small-en-v1.5) → vector search → **S2** rerank (bge-reranker-base) → **S3** generation (Qwen2.5-1.5B-Instruct, INT4). Each model is loaded once on one device; there is no per-request routing. Phase 1 only measures. It answers three questions: can every stage run on every device, how fast and at what cost, and why do devices differ.

Main findings.

- All 9 stage×device combinations run, and a placement check (GPU cycle counters, NPU busy time) confirms nothing silently falls back to another device.
- The iGPU is the fastest device for S2 and S3 at every size we measured. The NPU is competitive only for small encoder inputs (S1 batch 1; S2 with 10 candidates) and degrades sharply for larger shapes. The CPU is 15–20× slower for S2 and for S3 prefill.
- End to end, all stages on the iGPU is best for latency, throughput up to ∼2.4 queries/s, and answer quality. Moving S2 to the NPU adds 110–140 ms per query when S3 is on the iGPU or CPU (nothing when S3 is also on the NPU), lowers recall@3 from 0.87 to 0.82–0.83, and gives at most a few percent more capacity.
- Placing S1 on any device changes end-to-end latency by about 20 ms or less; S1 is irrelevant to the decision.
- Running two devices at once slows both, because they share memory bandwidth: S3 decode on the iGPU slows by 58% while an NPU reranker runs.

2 Setup

Machine. One Intel Core Ultra 7 356H machine (16 cores, Arc iGPU, "Intel AI Boost" NPU, about 93 GiB shared DRAM), Ubuntu 24.04, kernel 7.0, NPU driver 1.33.0. Runtime: OpenVINO 2026.4.1 and openvino-genai 2026.4.1.0, the only stack we found that covers CPU, iGPU and NPU. The machine is shared with other users' desktop sessions; load average was 1–8 during runs and is recorded in every result row.

Models. S1 and S2 are FP16 OpenVINO exports with static shapes (S1 $[b, 64]$, S2 $[k, 256]$, padded and truncated), because the NPU requires static shapes; the same shapes are used on all devices. S3 is INT4 with symmetric, channel-wise weights; a first asymmetric export produced garbage on the NPU. Decoding is greedy, at most 32 new tokens in the pipeline.

Data. A closed corpus of 25,880 Wikipedia passages (SQuAD train/dev and HotpotQA). Passage embeddings are computed offline and are not timed. Search is exact cosine similarity in numpy on the CPU (≈4 ms) and is not part of placement. Phase 1 uses only the first 100 of 500 SQuAD-dev questions (20 candidates, top-3 passages to S3); 500 HotpotQA questions are held out for Phase 2.

Protocol. Stage sweeps change one variable and report warm medians over 10 repetitions (S3: 5 on the iGPU, 3 on CPU and NPU) after one cold run. End-to-end runs process the 100 questions sequentially, or overlapped (S1/S2 of query i+1 with S3 of query i), or under Poisson or closed-loop arrivals; S3 uses a fresh prompt per repetition so the GenAI prefix cache cannot inflate results.

Harness modes. (1) *Stage sweeps*: S1 batch (1/8/32), S2 candidates (10/20/50/100, real candidates), S3 prompt (256/1k/4k) and output length (32/128/256). (2) *End to end*: 100 questions under a fixed S1/S2/S3 mapping,

Table 1: Support matrix (Phase 1b). Load = model load plus compile, with a warm compile cache / true cold. Activity evidence: iGPU cycles were zero on every non-iGPU run and NPU busy time was zero on every non-NPU run; "busy" is the share of the call's latency during which the NPU counter advanced.

Stage	Device	Fallback	Activity evidence	Load (s)
S1	CPU	none	no iGPU/NPU activity	0.39 / 0.39
	iGPU	none	8.8×10^4 cycles/call	0.44 / 1.0
	NPU	none	NPU busy 92%	0.07 / 1.0
S2	CPU	none	no iGPU/NPU activity	3.3 / 3.3
	iGPU	none	3.6×10^6 cycles/call	3.1 / 3.8
	NPU[a]	none	NPU busy 91%	0.12 / 68
S3	CPU	none[b]	no iGPU/NPU activity	0.67 / 0.67
	iGPU	none[b]	3.7×10^7 cycles/call	1.6 / 3.0
	NPU[c]	none[b]	NPU busy 95%	4.9 / 10.7

[a]Static shapes only: 50 candidates take 241 s to compile and 100 candidates 21 min, then run at 14.2 s per call (iGPU 0.52 s).
[b]`LLMPipeline` does not report `EXECUTION_DEVICES`, so placement rests on the activity counters. [c]Needs symmetric INT4 weights (an asymmetric INT8 export compiles but generates garbage); load shown for a 1152-token maximum prompt.

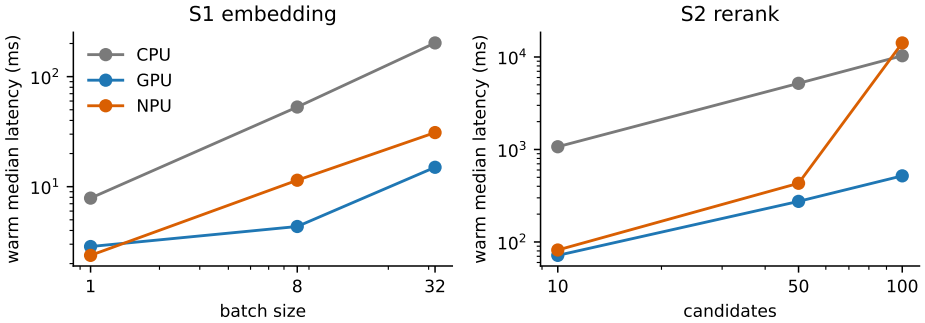


Figure 1: Warm median latency of S1 and S2 (log–log), first Phase 1 sweep; Table 2 uses the placement-verified Phase 1b rerun, which agrees within a few percent except S1 on the iGPU at batch 1. S2 on the NPU needs 4 min (50 candidates) and 21 min (100 candidates) to compile.

sequential or overlapped, with per-stage times, EM/F1 and recall. (3) *Load*: closed-loop clients (1/2/4) or Poisson arrivals (0.5–3 q/s); response time runs from arrival to completion. (4) *Contention*: S3 decode while an S2 loop runs on another device, in a thread or in a separate process.

Instrumentation (no root needed). *Placement*: `EXECUTION_DEVICES` for S1 and S2; for all stages, per-process DRM fdinfo (iGPU compute-engine cycles) and the global NPU busy counter, sampled around each measured call. *Memory*: a 20 ms sampler records peak process RSS, system memory, iGPU GTT memory and NPU memory; model footprint is the load peak minus the baseline. *Transfer*: S1/S2 calls are split into tokenize, set input, infer and get output, and infer is repeated with zero-copy device-context tensors, so the difference estimates the plugin's input copy; S3 uses GenAI's tokenize/inference/detokenize durations. *Cold start*: `NPU_BYPASS_UMD_CACHING=YES` bypasses the NPU driver's compile cache and `NEO_CACHE_PERSISTENT=0` disables the iGPU kernel cache.

3 Support matrix and actual placement

All nine stage–device pairs run and the counters agree with the requested device in every case, so there is no silent CPU fallback, including for the LLM, where the runtime does not report placement. The NPU is the least flexible device: it needs static shapes, including a maximum prompt length for the LLM, and compile cost grows quickly with shape. The NPU compile cache hides this after the first load (reranker load 0.12 s warm, 68 s cold); iGPU cold loads cost only 0.5–1.4 s more.

4 Per-stage results

S1. All devices take under 10 ms at batch 1. At batch 1 the iGPU and NPU are close (iGPU 1.4–2.9 ms depending on the run, NPU 2.4 ms) and fixed overhead dominates: 54–65% of the 64-token input is padding, yet shortening it

Table 2: S1 and S2 warm median latency (ms), Phase 1b run, 10 repetitions.

	S1, batch			S2, candidates			
	1	8	32	10	20	50	100
CPU	7.8	52.8	202	1076	2092	5180	10319
iGPU	1.4	4.2	15.1	71	129	276	522
NPU	2.4	9.0	27.9	84	173	434	14200

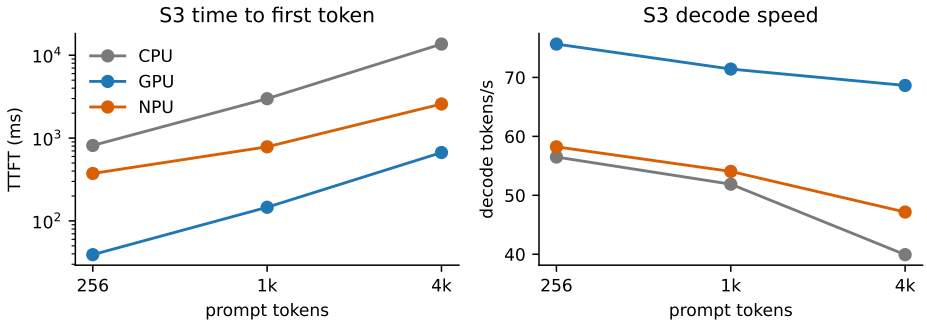


Figure 2: S3 on synthetic prompts built from corpus text (128 output tokens, fresh prompt per repetition; Oct 27 fresh-prompt sweep, the text quotes the Phase 1b rerun, within 3%). The NPU was compiled for `MAX_PROMPT_LEN`=4224.

barely changes latency. The iGPU scales much better (15 ms at batch 32 versus 28–31 ms on the NPU), and the CPU scales worst (202 ms). **S2.** The iGPU is the best choice from 10 candidates up. The NPU matches it only at 10 candidates, is 1.6× slower at 50, and collapses at 100 (14 s). S2 cost is also linear in sequence length (ad-hoc check, 20 candidates at 128/192/256 tokens: iGPU 48/73/99 ms). S2 padding is substantial: about 30% of rerank compute is padding and 12.6% of query–passage pairs are truncated at 256 tokens, which motivates length-bucketed batching in Phase 2.

S3. Prefill is the large difference (Fig. 2): at 1k tokens TTFT is 146 ms on the iGPU, 771 ms on the NPU and 2932 ms on the CPU. Decode speed is 69–75 tokens/s on the iGPU, 47–58 on the NPU and 40–56 on the CPU; decode is memory-bandwidth bound, so the gap is much smaller than in prefill. On the iGPU, output length does not change TTFT or time per output token (TPOT 13.3–14.6 ms), so total time is linear in output tokens. The NPU's TTFT depends on the compiled prompt limit rather than the real prompt: a 256-token prompt takes 99.6 ms with `MAX_PROMPT_LEN`=384 but 366 ms with 4224, and larger limits also use more device memory (Table 3).

An uncached NPU compile also needs 1.7–3.2 GB of host RAM, and a whole end-to-end process uses 1.7–2.4 GB of RSS, so memory is not a constraint on this machine, but the NPU LLM's footprint grows with its static prompt shape.

Data movement and placement. Because the devices share DRAM, "transfer" is a host-side copy, not a bus crossing, and the payload between stages is small (per query: 1.5 KB query vector, 14.6 KB of candidate text, 2.4 KB prompt). Input transfer is also cheap: the iGPU reranker's input copy is 3–13 ms and the NPU's is under 0.4 ms. Reranker tokenization (8–35 ms, 10–15% of iGPU S2) is a larger host-side cost; S3 tokenization and detokenization take under 2 ms. The first inference after loading is also slower: 8–12 ms extra for S1 on the iGPU, and up to 2× the warm latency for S2 on the NPU (199 versus 84 ms at 10 candidates). On NPU runs the NPU is busy for 91–95% of

Table 3: Cost beyond latency (Phase 1b, placement-verified runs). Load time is the cold start with caches bypassed; memory is added by loading the model.

Stage	Device	Load (s) cached / cold	Memory (MB)	Notes
S1 (b=1)	iGPU	0.44 / 0.98	130	
S1 (b=1)	NPU	0.07 / 0.98	67	
S2 (20 cand.)	iGPU	3.1 / 3.8	596–818	
S2 (20 cand.)	NPU	0.12 / 68	726	242 s for 50 cand.
S3	iGPU	1.6 / 3.0	1002	
S3	NPU	4.9 / 10.7	1355 / 1893 / 2030	for `MAX_PROMPT_LEN` 384 / 1152 / 4224; cached load 2.0 / 4.9 / 6.2 s
S3	CPU	0.7 / 0.7	(host RAM)	

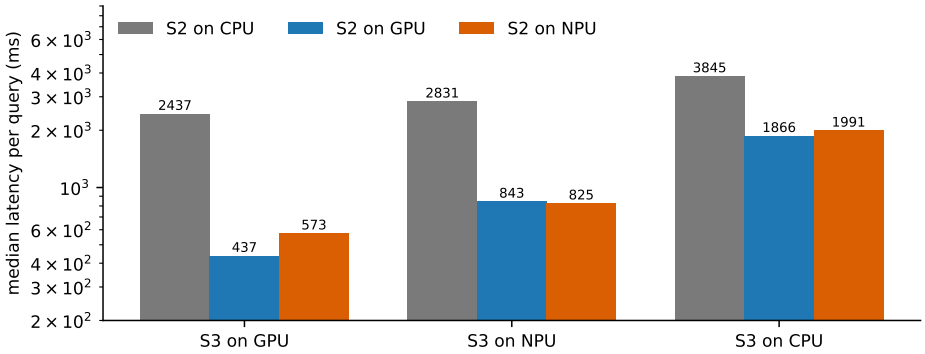


Figure 3: Median latency per query, sequential, 20 candidates, top-3 passages, S1 on the iGPU (the S1 device changes latency by about 20 ms or less).

Table 4: All 27 mappings were measured; S1 is shown on the iGPU (S1 on CPU or NPU changes latency by at most 22 ms in every row, see `coverage_full.csv`). Sequential, 100 dev questions per round. n = questions measured.

S2	S3	Median (ms)	p90 (ms)	q/s	recall@3	F1	n
iGPU	iGPU	**437**	621	**2.23**	0.87	0.36	300
NPU	iGPU	573	757	1.73	0.82	0.34	300
CPU	iGPU	2437	2624	0.41	0.87	0.36	300
iGPU	NPU	843	1093	1.19	0.87	0.38	200
NPU	NPU	825	1103	1.19	0.82	0.32	100
CPU	NPU	2831	3091	0.35	0.87	0.38	100
iGPU	CPU	1866	2445	0.52	0.87	0.34	100
NPU	CPU	1991	2423	0.51	0.82	0.32	100
CPU	CPU	3845	4384	0.26	0.87	0.34	100

the latency; on iGPU runs only GPU cycles accumulate, so the labels in our tables are the devices that actually ran.

5 End-to-end results

With S3 on the iGPU, a query takes 437 ms: S1 2 ms, search 4 ms, S2 118 ms, S3 316 ms (TTFT 81 ms). Putting S3 on the NPU doubles the latency, and on the CPU it multiplies it by 4. S2 on the NPU does not help sequentially (231 ms versus 118 ms) because the NPU reranker is slower than the iGPU's; with overlapped execution, where S2 of the next query runs while S3 decodes, S2 on the NPU reaches 2.45–2.50 queries/s versus 2.38 for all-iGPU, a gain of about 5%. All-iGPU gains only 7% from overlap (2.23 to 2.38) because both stages share one device, whereas S2 on the NPU gains 44% over its own sequential run (1.74 to 2.50); S3 on the iGPU then slows from 316 to about 385 ms (Sec. 5.2). Quality is a function of the S2 device: S2 on the CPU and the iGPU give identical rankings, but the NPU's FP16 numerics reorder the rerank scores and recall@3 falls from 0.87 to 0.82–0.83 (F1 0.36 to 0.34). The S3 device also changes the generated text slightly (F1 0.36 on the iGPU, 0.38 on the NPU, 0.34 on the CPU, from the same INT4 weights); with 100 questions these differences are not statistically tested and should not be read as meaningful.

5.1 Load

Every mapping behaves like a queue with a hard capacity: below it, response time is close to the service time; above it, waiting grows without bound (p50 of 42 s at 2.0 q/s for S2 and S3 on the NPU). Capacity is 2.4 q/s for S3

Table 5: Poisson arrivals: response time p50 / p95 (s) and capacity (highest achieved q/s over the offered rates).

S2	S3	1.0 q/s	1.5 q/s	2.0 q/s	2.5 q/s	Capacity
iGPU	iGPU	0.61 / 1.40	0.73 / 1.91	1.03 / 2.73	3.2 / 9.4	2.39
NPU	iGPU	0.73 / 1.51	0.83 / 1.95	1.03 / 2.57	2.9 / 8.2	2.40
iGPU	NPU	1.26 / 3.20	5.3 / 9.2	21 / 39	–	1.40
NPU	NPU	2.0 / 4.6	11.5 / 25	42 / 60	–	1.22

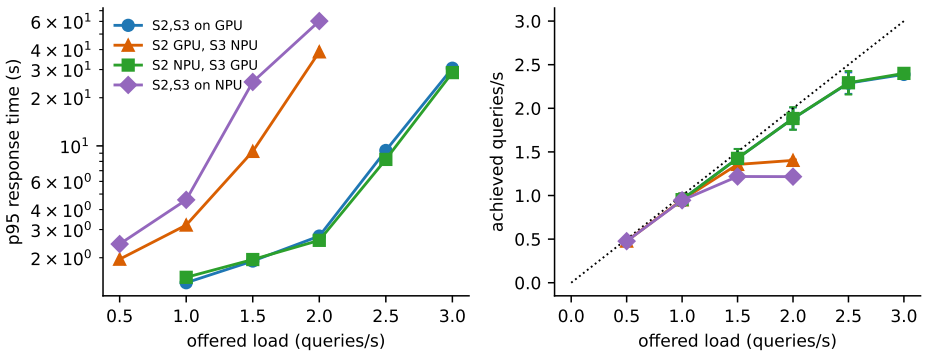


Figure 4: Open-loop Poisson arrivals (3 rounds, at least 150 queries per point; error bars: std over rounds). Response time includes queueing.

Table 6: S3 decode (1k-token prompt, 128 output tokens) with an S2 reranker running elsewhere.

S3 device	Aggressor	Mode	TTFT (ms)	TPOT (ms)
iGPU	none	–	148	14.0
iGPU	rerank on NPU	thread / process	191	22.1
iGPU	tokenize only	thread	157	14.2
iGPU	rerank on CPU	process	162	15.2
NPU	none	–	769	18.6
NPU	rerank on iGPU	process	1052	31.5

on the iGPU and 1.2–1.4 for S3 on the NPU. Between the two S3-on-iGPU mappings the evidence is mixed: in closed-loop runs S2 on the NPU gives 2.45–2.49 q/s versus 2.38 (+4%), but under Poisson arrivals the two have the same capacity within noise (std 0.12–0.14 q/s), the same p50 at 2.0 q/s, and a 12% lower p95 at 2.5 q/s for the NPU variant. At 1 q/s all-iGPU has the lower median (0.61 versus 0.73 s). Hence any benefit of an NPU reranker appears only near saturation and is small.

5.2 Why concurrent devices interfere

We ran S3 decode while a reranker loop ran on another device (Table 6). The slowdown is the same whether the aggressor is a thread or a separate process, and a thread that only tokenizes causes none, so it is not the Python GIL: the devices contend for shared memory bandwidth. An NPU reranker raises the iGPU's TPOT from 14.0 to 22.1 ms (+58%) and a GPU reranker raises the NPU's TPOT from 18.6 to 31.5 ms (+70%). A CPU reranker costs the iGPU only 8% (15.2 ms). The aggressor slows too (NPU rerank 4.4 calls/s versus 5.8 alone): both accelerators stream weights and activations from the same DRAM, and decode is bandwidth bound. This explains why overlapped pipelines gain much less than the sum of their parts.

5.3 Prompt size

The synthetic S3 sweep is for device characterization; real RAG prompts depend on how many passages are passed (Table 7). Top-1 prompts average 213 tokens, top-3 555 and top-8 1379 (max 1888). More passages raise recall@N from 0.74 to 0.90 but F1 falls from 0.40 to 0.25 and EM from 0.22 to 0.04, so the 1.5B model degrades with longer context, or answers more verbosely under our 32-token limit; we have not analyzed which. For latency, top-8 costs 764 ms on the iGPU (TTFT 247 ms) and 1342 ms on the NPU. Output length is not controlled in these runs, which is why top-1 is not faster than top-3.

6 Implications for Phase 2

1. **All-iGPU is a strong baseline.** It wins on latency, quality and capacity in almost every cell. A policy only has something to win in narrow regions: near-saturation load (S2 on the NPU), or when the iGPU is occupied by other work. Phase 2 should quantify exactly where a load-aware policy beats the fixed all-iGPU choice rather than assume it does.

Table 7: Top-N passages given to S3 (S1, S2 on the iGPU, 100 questions).

S3	N	Prompt tokens	TTFT (ms)	Median (ms)	recall@N	F1	EM
iGPU	1	213	71	517	0.74	0.40	0.22
iGPU	3	555	81	437	0.87	0.36	0.14
iGPU	8	1379	247	764	0.90	0.25	0.04
NPU	1	213	430	696	0.74	0.46	0.27
NPU	3	555	430	845	0.87	0.38	0.16
NPU	8	1379	828	1342	0.90	0.26	0.04

Table 8: Which device under which condition (Phase 1 conclusion).

Stage	Preferred device	Conditions and exceptions
S1	iGPU, but any device works	Under 10 ms per query everywhere; avoid the CPU for batches of 8 or more.
S2	iGPU alone; NPU only when overlapping S3	NPU only up to 50 candidates with a precompiled shape, and it costs about 4 points of recall@3; never the CPU.
S3	iGPU	NPU is about 1.9× slower end to end, with TTFT tied to its compiled maximum prompt; CPU only as a fallback.
Pipeline	All-iGPU; S2 on the NPU may help only near saturation	Same Poisson capacity (≈2.4 q/s) within noise; S2 on the NPU gives +4% closed-loop throughput and 12% lower p95 at 2.5 q/s; iGPU+NPU overlap costs up to +58% S3 TPOT.

2. **Candidate policy inputs** we can now model: S2 latency as a function of candidates and device; S3 TTFT and TPOT as functions of prompt length, with the NPU depending on its compiled limit; per-model memory and compile cost; and a memory-contention penalty (about +60% decode time when two devices stream at once).
3. **Cheap improvements to test:** length-bucketed S2 batches (up to ≈30% less rerank compute), choosing the NPU `MAX_PROMPT_LEN` per workload class, and the number of passages passed to S3.
4. **Remaining measurements:** the HotpotQA held-out set (multi-hop retrieval changes recall and prompt sizes), a mixed short/long workload, statistical tests on the quality differences, and a 3B model to check whether conclusions change when the iGPU is under more pressure.

7 Limitations and issues found

- **Invalid first S3 sweep.** Our first S3 sweep reused one prompt per configuration, so CPU and iGPU warm TTFT measured prefix-cache hits (for example 77 ms for a 4k prompt on the CPU). It was rerun with a fresh prompt per repetition; the numbers here come from the rerun.
- **Two NPU models in one process.** In one process, an NPU LLM together with any other NPU model fails: whichever was loaded first raises `zeMemAllocHost` (null handle) at its first inference. Two NPU encoders together are fine. The same code ran all-NPU on Oct 27 but fails now with unchanged OpenVINO and driver versions; we do not know why. For those mappings the LLM runs in its own process (one queue round trip of 0.4–0.6 ms per call). A control with S3 on the NPU in its own process (842 ms) versus in the same process (845 ms) shows no measurable overhead. The Oct 27 all-NPU reference (814 ms) agrees with the new measurement (815 ms).
- **Noise and sample size.** The machine is shared (load average 1–8). Only the main 9 S3-on-iGPU mappings have three rounds; the others have one or two. Quality comparisons use 100 questions with no confidence intervals. The S3 sweep on CPU and NPU covers only 128 output tokens, and its prompts are synthetic.
- **Scope.** The corpus is small (25,880 passages), so search is negligible; with a much larger index it would matter. Energy was not measured because power counters need root access, so our comparison is performance only.